

Behavioral Design Pattern Recognition Guide

CSE 314 — How to identify State, Strategy, Observer, Mediator, and Template Method from a word problem, plus ready-to-adapt Java templates.

Part 1 — Why Every Problem Looks the Same

Almost every problem in this course describes: **something that changes**, **something reacting to a change**, or **a fixed sequence of steps**. That surface similarity is intentional — the exam is testing whether you can read past the story (lock, ride, elevator, hospital, stock) to the underlying shape of the interaction. There are really only three shapes behavioral patterns solve:

Shape	What's happening structurally	Pattern(s)
One thing changes behavior over time	The same object behaves differently depending on an internal condition, and that condition itself changes as a result of operations	State
One thing needs interchangeable algorithms	The same operation can be done in multiple ways, chosen at runtime, without changing the caller	Strategy
One thing broadcasts, many things react	A single source of truth changes, and an unknown/variable number of dependents must be kept in sync automatically	Observer
Many things must not talk to each other directly	Multiple peer objects need to coordinate, but direct references between them would create a tangled web	Mediator
One process, fixed order, swappable details	Every case follows the identical sequence of steps, but a few steps differ per case	Template Method

Part 2 — The Recognition Procedure

Work through these questions **in order**. Stop at the first one that fires — most problems resolve in steps 1–3. If two questions fire together, you likely have a combination problem (very common in this course).

Step 1 — Underline the nouns and verbs that repeat.

List the entities in the problem (Lock, Ride, Order...) and the operations on them (enterPin, accept, setPrice...). This tells you the object whose behavior you must model.

Step 2 — Ask: "Does this object have a fixed set of named modes, and does its behavior/allowed-operations change per mode?"

Look for words like: states, statuses, phases, modes, stages, or a described lifecycle ("goes from X to Y to Z"), or "only operation A is allowed while in condition B". If yes → **State**. Giveaway vocabulary: Locked/Unlocked, Requested/Accepted/OnTrip, Placed/Confirmed/Shipped, Red/Yellow/Green, Idle/MovingUp/MovingDown, "invalid transitions should be rejected".

Step 3 — Ask: "Is there ONE task that can be done in multiple interchangeable ways, chosen by the user/caller?"

Look for: "customer selects an option", "choose a method", "multiple algorithms", "switch between X, Y, Z at runtime", "new options should be pluggable later". The key test: swapping the way does NOT change the rest of the object's behavior or its lifecycle — it's a one-shot computation/action, not a sequence of internal modes. If yes → **Strategy**. Giveaway vocabulary: payment methods, shipping options, compression algorithms, sorting order, notification channel, alert condition.

Step 4 — Ask: "When ONE thing changes, do MANY independent things need to find out automatically?"

Look for: "notify", "subscribe/unsubscribe", "all followers/displays/widgets update", "the publisher should not know the concrete type of its listeners", "add a new kind of listener later without changing the source". If yes → **Observer**. Giveaway vocabulary: WeatherStation + displays, StockData + widgets, price alerts, RavenBoard, notify all subscribers.

Step 5 — Ask: "Are there multiple peer objects that must NOT reference each other directly, but need a central object to coordinate them?"

Look for: "must not talk to each other directly", "a central controller/tower/hub decides", "coordinates access to a shared resource" (a runway, an elevator bank), "queues a request until a resource is free". If yes → **Mediator**. Giveaway vocabulary: ControlTower, Hub, ElevatorController, "queues the request", "grants/holds".

Step 6 — Ask: "Do ALL cases follow the exact same ordered list of steps, with only a few steps differing per case?"

Look for: a numbered procedure that is explicitly said to be "the same for every type", followed by "but each type customizes steps X and Y", or "must live in exactly one place" / "must not be duplicated". If yes → **Template Method**. Giveaway vocabulary: export formats sharing gather/finalize, hospital departments sharing check-in/vitals/discharge, import pipelines sharing read+validate then sort+summary.

Step 7 — Check for a second pattern layered on top.

Re-read the problem once more looking specifically for: (a) a lifecycle described with State language PLUS a "notify observers" requirement → State + Observer; (b) a Strategy choice that itself needs to be evaluated by many subscribers → Observer + Strategy; (c) a Mediator coordinating objects that themselves have State; (d) a Template Method whose one varying step is explicitly "pluggable"/"interchangeable" → Template Method + Strategy. This is exactly the pattern the practice sets use (S2, O2, M2, T2).

Quick sanity check after choosing a pattern: say out loud what plays the role of each participant (e.g. for State: "the Context is ___, the States are ___, the trigger operations are ___"). If you cannot name every role from the problem text, you've likely picked the wrong pattern or missed a combination.

Part 3 — One-Glance Keyword Lookup

Pattern	Trigger phrases	Combine with
State	states/statuses/phases, lifecycle $X \rightarrow Y \rightarrow Z$, "invalid transitions rejected", "only op A allowed while in B", mode-dependent behavior	Observer (notify on change), Mediator (coordinated stateful peers)
Strategy	choose an option/algorithm/method, switch at runtime, "pluggable", interchangeable computations, rejected only inside one algorithm's rule	Observer (each subscriber evaluates via its own strategy), Template Method (one variable step)
Observer	notify, subscribe/unsubscribe, "publisher doesn't know listener types", broadcast to many, add new listener type later	State (notify on state change), Strategy (per-subscriber evaluation logic)
Mediator	"must not talk to each other directly", central controller/tower/hub, shared resource, queue + grant/hold	State (each coordinated peer has its own state machine)
Template Method	same steps for every case, "must live in exactly one place"/"not duplicated", a few steps customized per subclass, optional/skipable step	Strategy (one step is swappable independently of the subclass)

Part 4 — Reusable Java Templates

Each template below is a minimal, compiling skeleton. Rename the placeholder classes/methods to match the problem, keep the structural shape (who calls whom) exactly as shown.

STATE PATTERN

Roles: Context (holds current state, delegates operations) · State interface (declares every operation) · Concrete States (one class per named state; each decides the next state or rejects the operation).

```
// ---- Template: State Pattern ----
interface State {
    void operationA(Context ctx);
    void operationB(Context ctx);
    String name();
}

class ConcreteStateA implements State {
    public void operationA(Context ctx) {
        // do the work valid for this state
        ctx.setState(new ConcreteStateB()); // valid transition
    }
    public void operationB(Context ctx) {
        System.out.println("Invalid in state A"); // reject, no transition
    }
    public String name() { return "StateA"; }
}

class ConcreteStateB implements State {
    public void operationA(Context ctx) {
        System.out.println("Invalid in state B");
    }
    public void operationB(Context ctx) {
        ctx.setState(new ConcreteStateA());
    }
    public String name() { return "StateB"; }
}

class Context {
    private State current = new ConcreteStateA(); // initial state
    public void setState(State s) { this.current = s; }
    public void operationA() { current.operationA(this); }
    public void operationB() { current.operationB(this); }
    public String currentStateName() { return current.name(); }
}

// Usage:
// Context ctx = new Context();
// ctx.operationA(); // triggers transition or is rejected, per state
```

STRATEGY PATTERN

Roles: Strategy interface (declares the one interchangeable operation) · Concrete Strategies (one class per algorithm/option) · Context (holds a reference to the current strategy and delegates to it; can swap it at runtime).

```
// ---- Template: Strategy Pattern ----
interface Strategy {
    void execute(/* parameters needed for the computation */);
}

class ConcreteStrategyA implements Strategy {
    public void execute() {
        System.out.println("Computed using algorithm A");
    }
}

class ConcreteStrategyB implements Strategy {
    public void execute() {
        System.out.println("Computed using algorithm B");
    }
}
```

```

    }
}

class Context {
    private Strategy strategy;
    public void setStrategy(Strategy strategy) { this.strategy = strategy; }
    public void performTask() { strategy.execute(); }
}

// Usage:
// Context ctx = new Context();
// ctx.setStrategy(new ConcreteStrategyA());
// ctx.performTask();
// ctx.setStrategy(new ConcreteStrategyB()); // swap at runtime
// ctx.performTask();

```

OBSERVER PATTERN

Roles: Observer interface (the update callback) · Concrete Observers (react to the update) · Subject/Publisher (keeps a list of observers, supports subscribe/unsubscribe, calls update on all of them when its state changes).

```

// ---- Template: Observer Pattern ----
import java.util.*;

interface Observer {
    void update(/* data describing what changed */);
}

class ConcreteObserverA implements Observer {
    public void update() {
        System.out.println("Observer A reacting to update");
    }
}

class ConcreteObserverB implements Observer {
    public void update() {
        System.out.println("Observer B reacting to update");
    }
}

class Subject {
    private List<Observer> observers = new ArrayList<>();

    public void subscribe(Observer o) { observers.add(o); }
    public void unsubscribe(Observer o) { observers.remove(o); }

    public void changeState(/* new data */) {
        // ... update internal data here ...
        notifyObservers();
    }

    private void notifyObservers() {
        for (Observer o : observers) o.update();
    }
}

// Usage:
// Subject subject = new Subject();
// Observer a = new ConcreteObserverA();
// subject.subscribe(a);
// subject.changeState(); // a is notified
// subject.unsubscribe(a);
// subject.changeState(); // a no longer notified

```

MEDIATOR PATTERN

Roles: Mediator interface (methods peers use to talk through it) · Concrete Mediator (owns the shared resource/coordination logic; decides who acts, queues when busy) · Colleagues/Peers (only ever call the mediator, never each other).

```
// ---- Template: Mediator Pattern ----
import java.util.*;

interface Mediator {
    void request(Colleague sender);
    void notifyFinished(Colleague sender);
}

class ConcreteMediator implements Mediator {
    private boolean resourceBusy = false;
    private Queue<Colleague> waiting = new LinkedList<>();

    public void request(Colleague sender) {
        if (!resourceBusy) {
            grant(sender);
        } else {
            System.out.println(sender.getName() + " must wait, resource busy.");
            waiting.add(sender);
        }
    }

    private void grant(Colleague c) {
        resourceBusy = true;
        System.out.println(c.getName() + " granted access.");
    }

    public void notifyFinished(Colleague sender) {
        resourceBusy = false;
        System.out.println(sender.getName() + " finished, resource freed.");
        if (!waiting.isEmpty()) {
            grant(waiting.poll());
        }
    }
}

class Colleague {
    private String name;
    private Mediator mediator;
    public Colleague(String name, Mediator mediator) { this.name = name; this.mediator = mediator; }
    public String getName() { return name; }
    public void makeRequest() { mediator.request(this); }
    public void finish() { mediator.notifyFinished(this); }
}

// Usage:
// Mediator mediator = new ConcreteMediator();
// Colleague c1 = new Colleague("C1", mediator);
// Colleague c2 = new Colleague("C2", mediator);
// c1.makeRequest();    // granted
// c2.makeRequest();    // waits, since resource busy
// c1.finish();         // frees resource, grants c2 automatically
```

TEMPLATE METHOD PATTERN

Roles: Abstract class with the *template method* (final; fixed order of steps) · shared/private steps (identical for everyone, live once in the base class) · abstract steps (must be overridden per subclass) · optional hook (has a default no-op body; subclasses override only if they need to).

```
// ---- Template: Template Method Pattern ----
abstract class Process {
    // Template method: fixed skeleton, cannot be overridden
    public final void run() {
        stepOneShared();
        stepTwoVariant();    // must be implemented per subclass
    }
}
```

```

        stepThreeVariant(); // must be implemented per subclass
        stepFourOptional(); // optional hook, default does nothing
        stepFiveShared();
    }

    private void stepOneShared() {
        System.out.println("Shared step 1 (identical for all)");
    }

    protected abstract void stepTwoVariant();

    protected abstract void stepThreeVariant();

    protected void stepFourOptional() {
        // default: no-op; override only if a subclass needs this step
    }

    private void stepFiveShared() {
        System.out.println("Shared step 5 (identical for all)");
    }
}

class ConcreteProcessA extends Process {
    protected void stepTwoVariant() { System.out.println("A's version of step 2"); }
    protected void stepThreeVariant() { System.out.println("A's version of step 3"); }
    protected void stepFourOptional() { System.out.println("A also does optional step 4"); }
}

class ConcreteProcessB extends Process {
    protected void stepTwoVariant() { System.out.println("B's version of step 2"); }
    protected void stepThreeVariant() { System.out.println("B's version of step 3"); }
    // B skips step 4 entirely - uses the base no-op
}

// Usage:
// new ConcreteProcessA().run();
// new ConcreteProcessB().run();

```

Part 5 — How Combination Problems Wire Together

In every combination problem from the practice sets, the two patterns plug in at one specific point. Recognize the seam, not just the two pattern names in isolation:

Combo	Where they connect
State + Observer (S2 Ride, M2 Elevator)	Inside <code>Context.setState()</code> , after swapping the state reference, call <code>notifyObservers()</code> . The State classes never touch the observer list directly — only the Context does, right after a transition.
Observer + Strategy (O2 Price Alerts)	Each Concrete Observer <i>owns</i> a Strategy field. Inside the observer's <code>update()</code> , it delegates the decision to <code>strategy.isTriggered(...)</code> instead of hardcoding logic. The Subject still only knows the Observer interface.
Mediator + State (M2 Elevator)	The Colleague objects (elevators) are themselves State-pattern Contexts. The Mediator calls plain methods on a Colleague (e.g. <code>moveTo(floor)</code>); internally that method drives its own state machine. The Mediator never inspects or sets a Colleague's state directly.
Template Method + Strategy (T2 Import Pipeline)	The abstract class's constructor (or a setter) takes a Strategy object and stores it as a field. One step inside the template method calls <code>strategy.execute(...)</code> instead of being abstract — so that step varies independently of which subclass is running.

Exam-time checklist: (1) Underline nouns/verbs. (2) Run Steps 2–6 in order, note every pattern that fires. (3) If more than one fired, find the seam from the table above. (4) Name every role out loud before writing code. (5) Copy the matching template(s) and rename.